

INDIAN STATISTICAL INSTITUTE

Project Linked Person — Selection 2026–2027

Subject: Systems Support (Software, Hardware and Web)

Part B — Practical Examination

Total: 16 + 18 + 16 = 50 marks Duration: 90 minutes

EXAMINATION INSTRUCTIONS

1. You have root on a virtual machine. It is a disposable clone: you cannot damage anything that matters, and it is connected to no real network. Log in with `ssh candidate@192.168.56.10`.
2. Each question has four parts — *the problem, how to reproduce it, what to fix, and how to check your work*. Run the checks yourself; you should not have to guess whether you are finished.
3. **The Constraints of every question are marked. Breaking a constraint loses marks even when the symptom goes away.** Examples: `chmod 777`, pinning names into `/etc/hosts`, reformatting a filesystem.
4. Keep `~/FIXLOG.md` up to date as you go: symptom, cause, what you changed, and how you verified it. It is marked, and there is a short oral discussion of your work afterwards.
5. You are not expected to know all of this in advance. Partial credit is real: a fault you diagnose correctly and explain, but cannot finish, is worth more than one you paper over.

NOTE: `man`, `--help`, `journalctl` and the logs under `/var/log` are all available on the machine. Your commands are recorded. Questions may be attempted in any order.

Q1. Disk full on the lab file server.

[16 marks]

Reported by a research scholar: “I cannot copy anything into `/srv/data` — it says the disk is full, and nothing I can see in there accounts for that much space. Submitting a job into `/srv/spool` also says the disk is full, but `df -h` shows that one has plenty of space left. That part makes no sense to me.”

Background: this server has three separate disks. `/srv/data` and `/srv/spool` look like ordinary folders, but each one is really its own small disk, attached at that path (a *mount point*). The rest of the system lives on a third, much larger disk.

<code>/</code>	the system disk	about 24 GB
<code>/srv/data</code>	the project data disk	1 GB
<code>/srv/spool</code>	the job-submission disk	48 MB

Each disk fills up on its own: one being full says nothing about the others. `df -h / /srv/data /srv/spool` prints one line per disk.

The two complaints look alike — both end in “No space left on device” — but they are **two unrelated faults on two different disks**, with different causes. Fixing one does

nothing for the other. Tackle them in either order; each is marked on its own.

(a) /srv/data is full [12 marks]

/srv/data is at 100%, and the files you can see there do not add up to the space reported as used.

Reproduce:

```
df -h /srv/data          # 100% used
sudo du -sh /srv/data    # much smaller than df says is in use
```

Fix: bring /srv/data back to **under 10% used** (the Use% column of `df -h`). There is more than one reason this disk is full: freeing the largest obvious files will not get you there on its own. Treat any difference between what `df` says is used and what `du` can account for as something you still have to explain.

Check:

```
df -h /srv/data          # Use% under 10%
ls /srv/data/current     # five survey-block CSV files, still there
```

(b) /srv/spool refuses new files [4 marks]

New files cannot be created on /srv/spool, *although df -h reports free space on it*. Something other than free space has run out. This has nothing to do with /srv/data.

Reproduce:

```
df -h /srv/spool         # shows space free
touch /srv/spool/incoming/testjob # "No space left on device"
```

Fix: make /srv/spool able to accept new files again. It is *not* short of space: freeing bytes there will not help. A filesystem can run out of more than one kind of resource, and `df` has another mode that reports the other one. Then, in ~/FIXLOG.md, explain in two or three sentences why /srv/spool reported “No space left on device” while `df -h` showed free space.

Check:

```
touch /srv/spool/incoming/t && echo OK && rm /srv/spool/incoming/t
```

Constraints — these apply to both parts, and are marked

- Nothing under /srv/data/current may be deleted or changed. It is live survey data with no second copy on this machine.
- Do not reformat, recreate, resize or unmount either disk.
- /srv/data/archive/README.txt says what is safe to remove there. Read it before deleting anything.
- Rebooting clears one of these faults. On a real file server at 11am you would not have that option; treat it as unavailable.

Q2. Shared project folder unusable. [18 marks]

Reported for the Statistics Lab group: “Bikram cannot save anything into `shared/`, and cannot open `notes.md` at all. When I create a file there, the others cannot edit it — it comes out belonging to my own personal group instead of the project group. `sudo systemctl restart reportd` used to work for all of us; now it is refused. And Chandan from the internal audit cell needs to *read* that folder for the annual review. He cannot see anything.”

Background: who is who. /srv/projects/statlab/ (and shared/ inside it) belongs to the statlab group. Anyone who is not a member or the auditor should have no access at all.

anita	member of statlab	read and write everything
bikram	member of statlab	read and write everything
chandan	auditor, not a member	read everything, write nothing

To see the current state:

```
cd /srv/projects/statlab
stat -c '%a %U:%G %n' . shared shared/notes.md
id bikram; getfacl shared; sudo cat /etc/sudoers.d/statlab
```

The complaints overlap, but underneath them are **five independent faults**, each with its own cause and fix. Fixing one does nothing for the others; each is marked on its own. Tackle them in any order. Test as the users named, not as root — root can do everything, so a check run as root proves nothing.

(a) Bikram is no longer in the statlab group [3 marks]

Reproduce: id bikram — statlab is missing. *Fix:* put him back, without changing any of his other groups.

```
id -nG bikram | grep -qw statlab && echo "bikram is a member"
```

(b) The shared folder: group write, and which group new files get [8 marks]

Members cannot create files in the folder, and once they can, new files come out in the creator's *personal* group instead of statlab. Both are about the permission bits on the two directories /srv/projects/statlab and shared/. Anita is still a member, so test as her.

Fix: both directories let the group read and write, give everyone else no access, and make new files inside them belong to statlab automatically. A directory can carry more than the nine permission bits you see first.

```
sudo -u anita touch /srv/projects/statlab/shared/t1 && echo "write OK"
stat -c '%U:%G' /srv/projects/statlab/shared/t1      # group must be: statlab
sudo rm -f /srv/projects/statlab/shared/t1
```

The stat line is the one people miss: if the group comes out as anita, this part is not fixed yet.

(c) notes.md has become private [2 marks]

shared/notes.md is the group's working file, but now belongs to one person and only she can read it. *Fix:* group statlab, readable and writable by the group, no access for anyone else.

```
stat -c '%a %G' /srv/projects/statlab/shared/notes.md  # 660 statlab
```

(d) The group's sudo right is broken [4 marks]

Members of statlab — and only them — should be able to run sudo -n systemctl restart reportd without a password. The rule is in /etc/sudoers.d/statlab and no longer does what it was meant to. *Fix:* correct the rule, editing the file only with sudo visudo -f /etc/sudoers.d/statlab — never a plain editor.

```
sudo -u anita sudo -n systemctl restart reportedd && echo "sudo OK"
sudo visudo -cf /etc/sudoers.d/statlab # parsed OK
```

(e) The auditor cannot read the folder [1 mark]

Chandan's access came from per-user entries on the files themselves, and those have been wiped. *Fix*: give him read-only access to everything under `/srv/projects/statlab/`, including files created there in future — *without* adding him to the group.

```
sudo -u chandan cat /srv/projects/statlab/shared/notes.md # must work
sudo -u chandan touch /srv/projects/statlab/shared/t2 # must FAIL
```

When all five are done

Bikram's original complaint needs (a), (b) and (c) together:

```
sudo -u bikram touch /srv/projects/statlab/shared/t3 # must work
sudo -u bikram cat /srv/projects/statlab/shared/notes.md # must work
```

Constraints — these apply to every part, and are marked

- **Chandan must not be put into the statlab group.** Audit policy forbids putting auditors into the groups they audit. He must be able to read and must *not* be able to write.
- No `chmod 777`, anywhere. This is a multi-user machine.
- Do not change anyone's login shell, password or home directory.

Hint. Group membership changes do not affect a shell that is already open; `sudo -u user` starts a fresh one.

Q3. The lab dashboard nobody can reach. [16 marks]

Reported by the Statistics Lab: “Our results dashboard has been down since Friday's maintenance. We open `http://gateway:8080` and get nothing. Somebody said the tunnel service on labbox keeps restarting. We need it back before the review meeting.”

Background: how the dashboard gets out of the basement. The dashboard runs on **labbox**, a basement machine. Campus IT lets labbox connect *out*, but nothing may connect *in*. So labbox publishes it through **gateway** (this VM) with a *reverse SSH tunnel*:

```
colleagues --> gateway:8080 ===== SSH tunnel =====> labbox --> 127.0.0.1:8888
              (this VM)      <-- labbox dials OUT --              (the dashboard)
```

labbox runs, as the service `lab-tunnel`:

```
ssh -N -R 0.0.0.0:8080:localhost:8888 tunnel@gateway
```

In `-R`, the first half (`0.0.0.0:8080`) is where *gateway* listens. The second half (`localhost:8888`) is where labbox delivers each connection, *as labbox sees it* — so that `localhost` is labbox's own.

gateway 10.20.0.1, this VM you are already here
labbox 10.20.0.2, basement `sudo labbox` opens its console

labbox is simulated on this VM and shares its files; its own are under `/opt/labbox/`. Its services are `labbox-dashboard` and `lab-tunnel`.

Friday's maintenance broke the tunnel in **three independent places**. Each has its own cause and fix and is marked on its own; fixing one does nothing for the others. Tackle them in any order — though you will only *see* (b) and (c) in action once (a) lets the tunnel log in.

(a) gateway refuses labbox's key **[5 marks]**

labbox logs in to gateway as `tunnel`, with a key and no password. gateway is rejecting it, so the tunnel never comes up and restarts every few seconds. The client's log says *that* it was denied; the server's says *why*.

```
journalctl -u lab-tunnel -n 5          # Permission denied (publickey...)
journalctl -u ssh -n 20                # gateway's side of the story
```

Fix: make gateway accept the key again. The key itself is correct; sshd refuses an `authorized_keys` file that others could have tampered with.

```
journalctl -u ssh -n 5                # Accepted publickey for tunnel
```

(b) The tunnel leads to the wrong place **[5 marks]**

The `-R` setting decides where on labbox each connection ends up. It must lead to the dashboard.

```
sudo labbox ss -tlnp                  # what is really listening on labbox
systemctl cat lab-tunnel              # where the tunnel sends connections
```

Fix: correct the tunnel in `/etc/systemd/system/lab-tunnel.service`, then `systemctl daemon-reload` and `systemctl restart lab-tunnel`.

```
systemctl cat lab-tunnel | grep -- -R # ends in localhost:8888
```

(c) gateway only lets itself use the tunnel **[6 marks]**

Once the tunnel is up, `curl http://localhost:8080` *on gateway* may work, yet everyone else is refused. Look at *which address* gateway is listening on for port 8080.

```
ss -tlnp | grep 8080                  # 127.0.0.1:8080 = loopback only
curl -s http://10.20.0.1:8080         # refused
```

Fix: by default sshd only lets a reverse tunnel listen on gateway's loopback, whatever the client asks for. The setting is in sshd's configuration for the `tunnel` user, `/etc/ssh/sshd_config.d/60-lab-tunnel.conf`. Change it, run `sudo sshd -t`, then `sudo systemctl reload ssh` and restart the tunnel.

```
ss -tlnp | grep 8080                  # 0.0.0.0:8080
```

When all three are done

```
curl -s http://10.20.0.1:8080 | grep STATLAB # STATLAB-DASHBOARD-OK
```

You can also open `http://192.168.56.10:8080` in a browser on your own machine — that is what the colleagues see.

Constraints — these apply to every part, and are marked

- **Do not turn off StrictModes** (or any other sshd safety check) to make the key work. Fix the file.

- **labbox must stay unreachable.** Do not loosen its firewall or move the dashboard off 127.0.0.1: going round the tunnel is not fixing it.
- Do not give the `tunnel` account a password or a login shell.
- Always run `sudo sshd -t` before reloading `sshd`. A broken `sshd` configuration can lock everyone out, you included.